

Memory

2026 Semester 1 SOFTENG 370: Operating Systems

Talia Xu

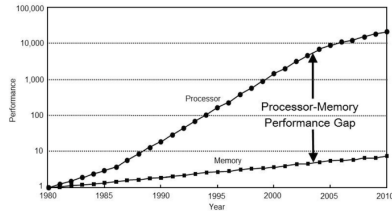
Lecture 10

2.0.0

Based on original slides by Professor Jon Eyolfson.

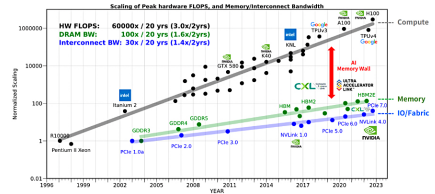
This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

The Memory Wall Problem



Computer Architecture: A Quantitative Approach by John L. Hennessy, David A. Patterson, Andrea C. Arpaci-Dusseau

The Memory Wall: BW Compute >> Memory >> IO/Fabric



UnifabriX

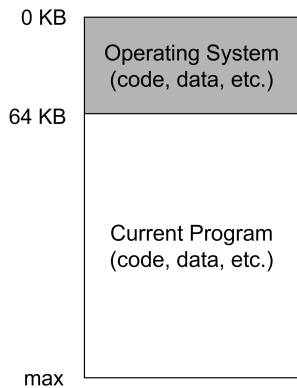
© 2019 UnifabriX. All rights reserved.

13

Memory is one of the most important topics in OS.

Hardware and software development are both very active (commercially and academically).

The early operating system



(Almost) No hardware abstraction.

OS is a set of routines (libraries).

A single running program in physical memory.

Multiple people start sharing a computer

⇒ Timeshare: One user runs for a bit, saves everything to disk, then load another.

But it is very slow.

Writing from disk to memory is slow...but (registers) are fast

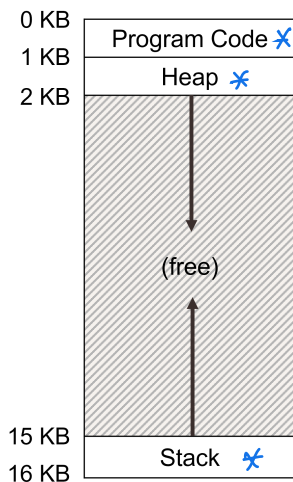
0 KB	Operating System (code, data, etc.)
64 KB	(free)
<u>128 KB</u>	<u>Process C</u> (code, data, etc.)
<u>192 KB</u>	<u>Process B</u> (code, data, etc.)
256 KB	(free)
<u>320 KB</u>	<u>Process A</u> (code, data, etc.)
384 KB	(free)
448 KB	(free)
512 KB	(free)

A second try - leaves everything in memory, but switch between them.

Each process gets 64 KB space from physical memory.

But there is no abstraction - any problems?

Create an abstraction – address space (virtual memory) .



(user) .

An address space is the running program's view of memory in the system.

Code: where instructions live

Heap: dynamic data (malloc'd data)

Stack: local variables, arguments to routines, re-turn values, etc.

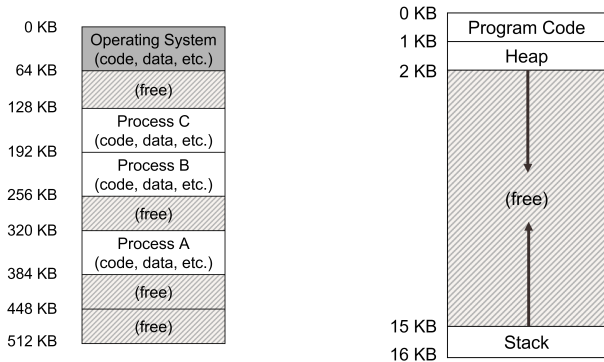
Create an abstraction – address space

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    printf("location of code : %p\n", main);
    printf("location of heap : %p\n", malloc(100e3));
    int x = 3;
    printf("location of stack: %p\n", &x);
    return x;
}
```

Virtual address or physical memory?

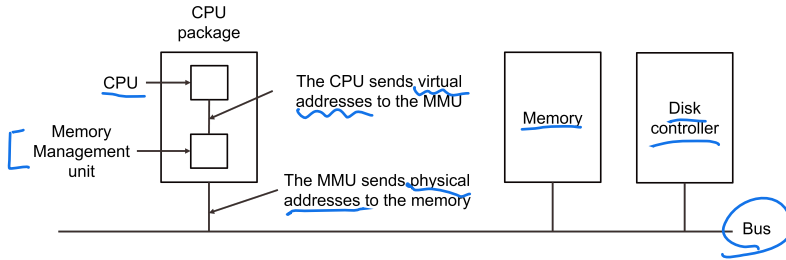
Create an abstraction – address space

Difference?



How can the OS build this abstraction of a private, potentially large address space for multiple running processes (all sharing memory) on top of a single, physical memory?

The memory management unit (MMU) *(hardware)*



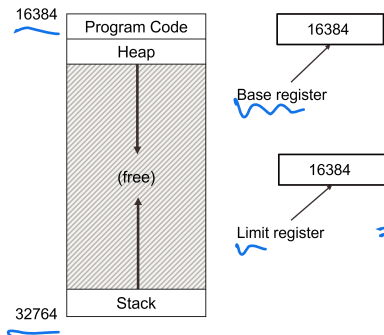
Virtual Memory Checklist

- Multiple processes must be able to co-exist
- Processes are not aware they are sharing physical memory
- Processes cannot access each others data (unless allowed explicitly)
- • Performance close to using physical memory
- Limit the amount of fragmentation (wasted memory)

How many of the following statements are correct?

1. A user-space process's virtual memory may be larger than the system's installed physical RAM. 2^{64} bytes. $\gg 1TB$. T.
2. For a process to execute, its entire virtual memory must be loaded into physical memory. F.
3. Two different processes can map the same physical address at different virtual addresses and read it concurrently. T.
4. The memory addresses generated by the CPU are physical memory addresses. F. (dynamic library).

Base and limit registers



The simple solution: Each CPU has two special hardware registers: base and limit.

When a process is run, the addresses are loaded to these registers.

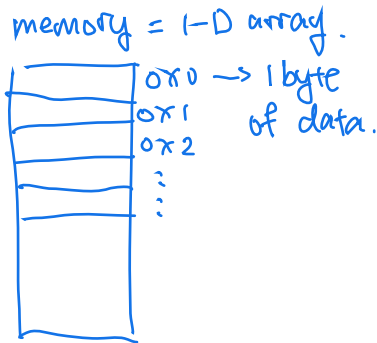
⇒ An addition and a comparison on every memory reference.

Remember That Memory is Byte Addressable

The smallest unit you can use to address memory is one byte

You can read or write one byte at a time at minimum

Each "address" is like an index of an array



Segmentation or Segments are Coarse Grained

Divide the virtual address space into segments for: code, data, stack, and heap

Each segment is a variable size, and can be dynamically resized

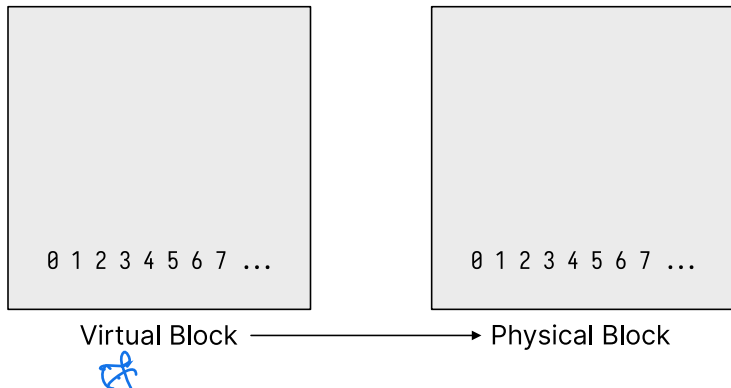
This is an old legacy technique that's no longer used

⇒ Segments can be large and very costly to relocate

It also leads to fragmentation (gaps of unused memory)

No longer used in modern operating systems

First Insight: Divide Memory into Fixed-Sized Chunks *(page)*.



Paging

One technique is to divide memory up into fixed-size pages (typically 4096 bytes or 4 KB)

- A page in virtual memory is called a page
- A page in physical memory is called a frame

You Typically Do Not Use All 64 Virtual Address Bits

$\Rightarrow$ GiB (base 2).

GB. (base 10)

CPUs may have different levels of virtual addresses you can use
Implementation ideas are the same

1 KB = 1 KiB.

= 1024 B.

We'll assume a 39 bit virtual address space used by RISC-V and other architectures

Allows for 512 GiB of addressable memory (called Sv39)

Implemented with a page table indexed by Virtual Page Number (VPN)
Looks up the Physical Page Number (PPN)

The Page Table Translates Virtual to Physical Addresses

Index + offset = 39 bits.
(8V39).

Offset:

position w/in a page.

12 bits $\rightarrow \log_2(\text{Page Size})$

$= \log_2(4096).$

$= 12 \text{ bits.}$

index:

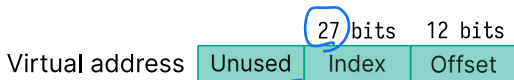
page #.

$= 2^{27}$

Size of Page table. $\times (8) 2^3$

row $\times$ # element $= 2^{30} \cdot 16$

(start)



VPN

2^{27} entries

stores the
translation
btw VPN $\rightarrow$ PPN

Page Table

64 bits.
(PTE)

44 bits

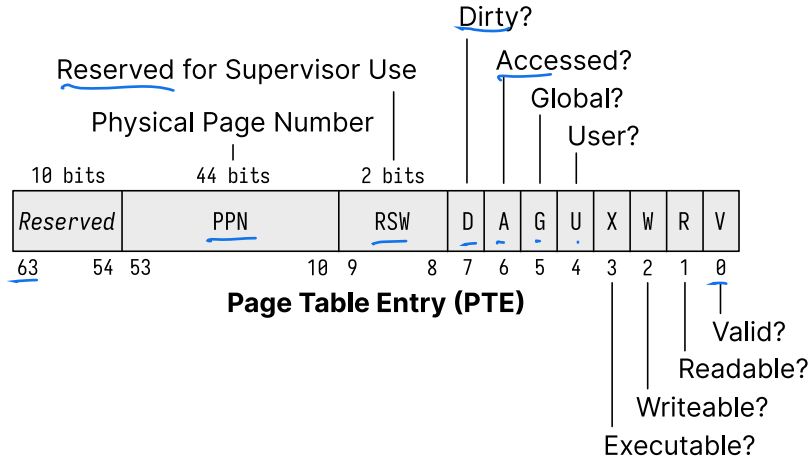
12 bits

(end).

Physical address



The Page Table Entry (PTE) Also Stores Flags in the Lower Bits



The Kernel Handles Translating Virtual Addresses

Considering the following page table:

VPN	PPN
0x0	0x1
0x1	0x4
0x2	0x3
0x3	0x7

4 entries.

page size.

4kB.



12 bits.

every hex # represents
up to 4 bits.

We would get the following virtual → physical address translations: 0x0 → 0xF

0x0AB0 →	0x0AB0.	
0x1FA0 →	index	offset
0x2884 →		
0x32D0 →		

↓

0x1AB0.

0b0 → 0b1111

12/4 = 3 hex #.

offset

Page Translation Example Problem

Virtual Addr	2	6
	index	offset.
Physical Addr.	4.	6

Assume you have a 8-bit virtual address, 10-bit physical address and each page is 64 bytes

- How many virtual pages are there? 4
- How many physical pages are there? 16
- How many entries are in the page table? 4
- Given the page table is [0x2, 0x5, 0x1, 0x8]

what's the physical address of 0xF1? 0x231

$$\# \text{ of PP} = 2^4 = 16$$

$$\log_2(\text{page size}) = \log_2(64) = 6$$

$$2^{(\# \text{ index})} = 2^2 = 4. \# \text{ VP.}$$



$$0xF1 \rightarrow 0b \underbrace{1111}_{\text{index}} \underbrace{0001}_{\text{offset.}}$$

0b11 → 3.

$$\Rightarrow 0b \underbrace{1000110001}_{\text{physical addr.}} = 0x231$$

Page Translation Example Problem

Assume you have a 8-bit virtual address, 10-bit physical address and each page is 64 bytes

- How many virtual pages are there? $\frac{2^8}{2^6} = 4$
- How many physical pages are there? $\frac{2^{10}}{2^6} = 16$
- How many entries are in the page table? 4
- Given the page table is [0x2, 0x5, 0x1, 0x8]
what's the physical address of 0xF1?
0x231

Each Process Gets Its Own Page Table

When you fork a process, it will copy the page table from the parent

Turn off the write permission so the kernel can implement copy-on-write

The problem is there are 2^{27} entries in the page table, each one is 8 bytes

This means the page table would be

Note that RISC-V translates a 39-bit virtual to a 56-bit physical address

It has 10 bits to spare in the PTE and could expand

Page size is 4096 bytes (size of offset field)

We Use Pages for Memory Translation

Divide memory into blocks, so we only have to translate once per block

Use page tables (array of PTEs) to access the PPN (and flags)

New problem: these page tables are always huge!